

Private Network Service Platform — Phase 1 Report

Oct 4, 2026

Computer Networks course project. Phase 1: Build and Observe. Team domain: **dimaginaxal.test**.

Member	Machine	Role in the network	IP address
Md. Sajjan	Mac 1	Private DNS server (dnsmasq) + test client	10.7.2.83
Sachin Jaiswal	Mac 2	Edge: nginx reverse proxy, load balancer, TLS	10.7.1.138
Sibtain Ahmed Qureshi	Mac 3	Backend A (Node.js, port 3001)	10.7.12.144
Viraj Chafale	Mac 4	Backend B (Node.js, port 3002) + test client	10.7.16.85

1. Summary

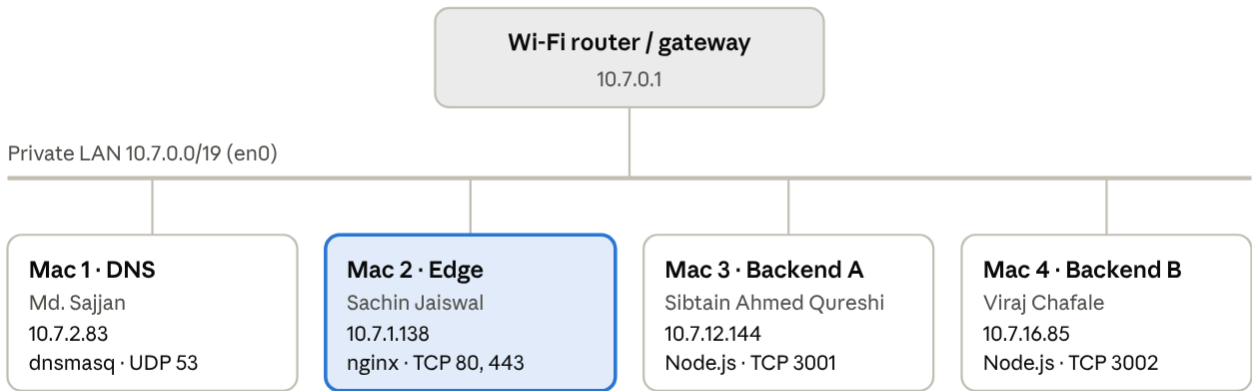
A client on our LAN can now open **<https://app.dimaginaxal.test>**, get the name resolved by our own DNS server, connect over trusted HTTPS to nginx, and get a reply from Backend A or Backend B in turn. Six repeated requests came back A, B, A, B, A, B.

Everything runs on four MacBooks on one Wi-Fi network (10.7.0.0/19). Nothing is hosted in the cloud. The backends are deliberately tiny JSON services, because the network setup is what this project is about.

We also captured the traffic in Wireshark on Mac 2. The captures show the DNS lookup for our domain and nginx opening a TCP connection to a backend and fetching `/api/status`. A few pieces of evidence are still to be captured before the review; they are listed in Section 11.

2. Network topology

Four Macs share one Wi-Fi subnet; only Mac 2 faces clients



Network topology · 4 Macs on one subnet

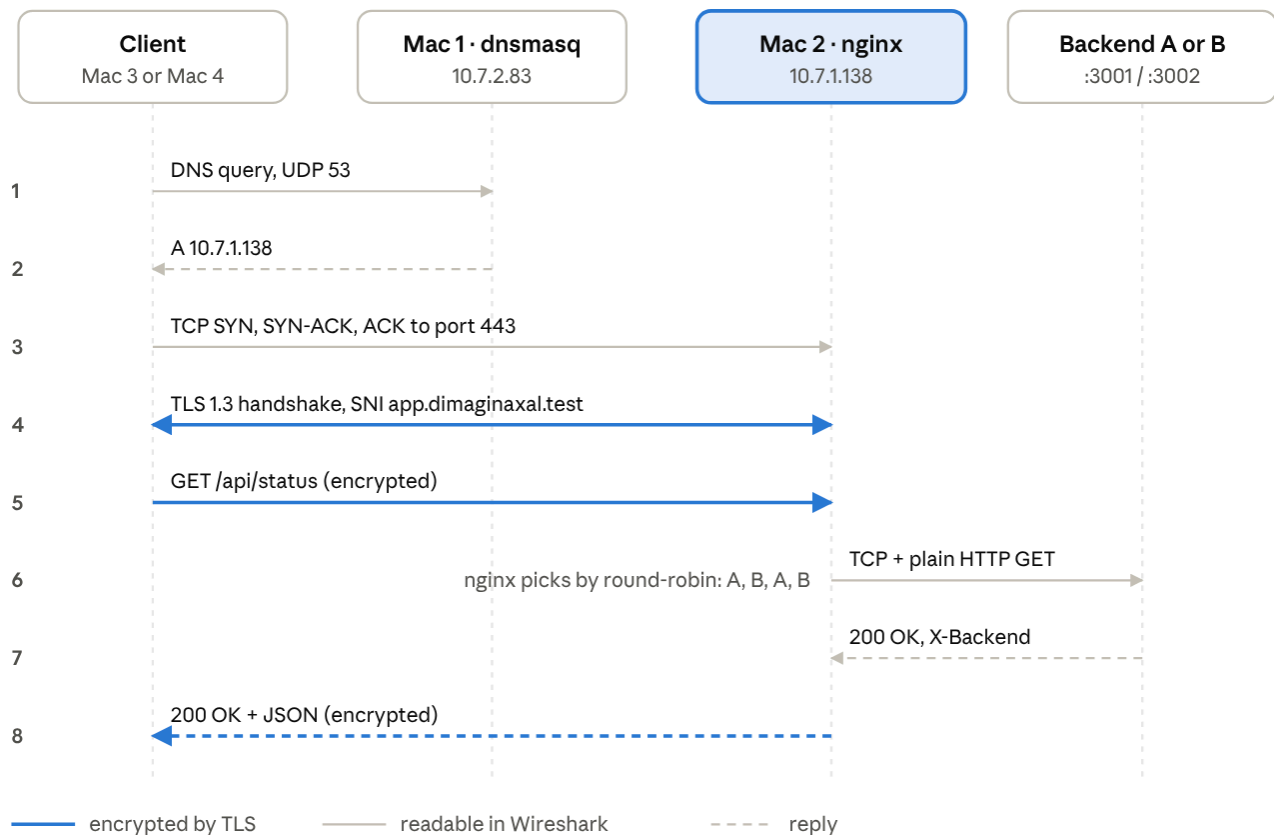
All four Macs hang off the same Wi-Fi network. Physically every Mac can reach every other one; logically, clients only ever connect to Mac 2.

Machine	Software	Listens on	Cloud equivalent
Mac 1	dnsmasq 2.93	UDP 53	Managed DNS (e.g. AWS Route 53 private zone)
Mac 2	nginx 1.31.6 + mkcert certificate	TCP 80 (redirect), TCP 443	Load balancer / CDN edge (e.g. AWS ALB)
Mac 3	Node.js HTTP server	TCP 3001	App server instance A
Mac 4	Node.js HTTP server	TCP 3002	App server instance B

Mac 1 and Mac 4 also act as test clients.

3. How one request travels

One request: DNS first, then TCP and TLS to nginx, then HTTP to a backend



Request flow · 8 messages, client to backend and back

Steps 1 and 2 find the address. Steps 3 to 5 open a secure channel to nginx. Steps 6 and 7 happen only inside the LAN, where nginx talks plain HTTP to whichever backend is next.

TCP/IP layer	OSI layer	What it is in our project
Application	7 (TLS sits around 5–6)	DNS, HTTP/1.1; TLS 1.3 wraps HTTP between client and nginx
Transport	4	UDP 53 for DNS; TCP 443, 3001, 3002
Internet	3	IPv4, all hosts in 10.7.0.0/19
Link	2 and 1	Wi-Fi frames with MAC addresses (Wireshark shows them as Ethernet II)

4. Task A: Private LAN

All four Macs joined the same Wi-Fi and got addresses in **10.7.0.0/19** (mask 255.255.224.0, gateway 10.7.0.1, interface en0 on every machine). A /19 gives 8,190 usable hosts, so all four addresses fall in one subnet even though they look far apart (10.7.1.x to 10.7.16.x).

Machine	IP	MAC address	Hostname
Mac 1	10.7.2.83	a6:09:ce:51:90:ce	Mds-MacBook-Pro.local
Mac 2	10.7.1.138	a2:32:d7:f5:7b:6f	Sachin-MacBook-Pro.local
Mac 3	10.7.12.144	16:82:84:67:cc:e5	Sibtains-MacBook-Pro.local
Mac 4	10.7.16.85	9e:9a:27:5b:54:3f	Virajs-MacBook-Pro.local

Ping results between the machines:

From → To	Packet loss	Note
Mac 1 → Mac 2	0%	
Mac 1 → Mac 3	25%	high, uneven latency
Mac 1 → Mac 4	0%	
Mac 2 → Mac 1	25%	
Mac 2 → Mac 3	0%	
Mac 2 → Mac 4	0%	

Every pair could reach each other, but some early runs lost packets. We think this is the shared Wi-Fi: the Mac 2 → Mac 1 path later carried all of Mac 2's DNS lookups without a single failed query in our capture. Ping from Mac 3 and Mac 4 as the source is not recorded yet.

5. Task B: Private DNS on Mac 1

Mac 1 runs **dnsmasq 2.93** on UDP port 53. It answers our two project names itself and forwards every other name to 1.1.1.1 or 8.8.8.8.

Name	Resolves to	Why
app.dimaginaxal.test	10.7.1.138	Mac 2 is the only public entry point

Name	Resolves to	Why
api.dimaginaxal.test	10.7.1.138	Same edge, second name

We used the reserved **.test** suffix. We avoided **.local** because macOS already uses it for Bonjour (mDNS), and lookups would go there instead of to our server.

Mac 2, Mac 3 and Mac 4 were pointed at Mac 1 as their only DNS server:

```
sudo networksetup -setdnsservers Wi-Fi 10.7.2.83
```

`dig @10.7.2.83 app.dimaginaxal.test` returned status NOERROR with the answer 10.7.1.138 from server 10.7.2.83#53. The same check passed for api.dimaginaxal.test.

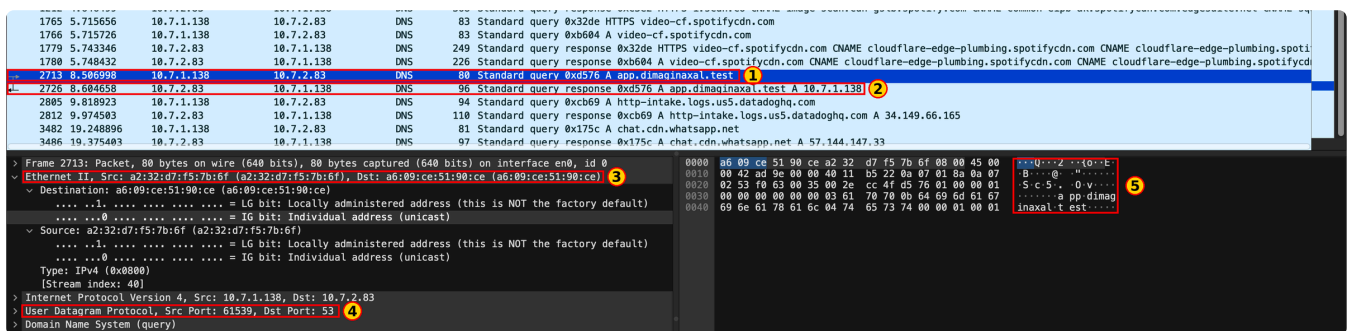


Figure 1. DNS query (frame 2713), captured on Mac 2, filter `dns`.

1. Mac 2 (10.7.1.138) asks Mac 1 (10.7.2.83) for the A record of app.dimaginaxal.test.
2. Mac 1 answers 0.1 s later. The transaction ID 0xd576 matches the query.
3. Ethernet source is Mac 2's MAC and destination is Mac 1's MAC. Both are on the same subnet, so the frame goes straight to Mac 1, not through the router.
4. DNS rides on UDP: client port 61539, server port 53.
5. The domain name is visible in the raw bytes. DNS is not encrypted.

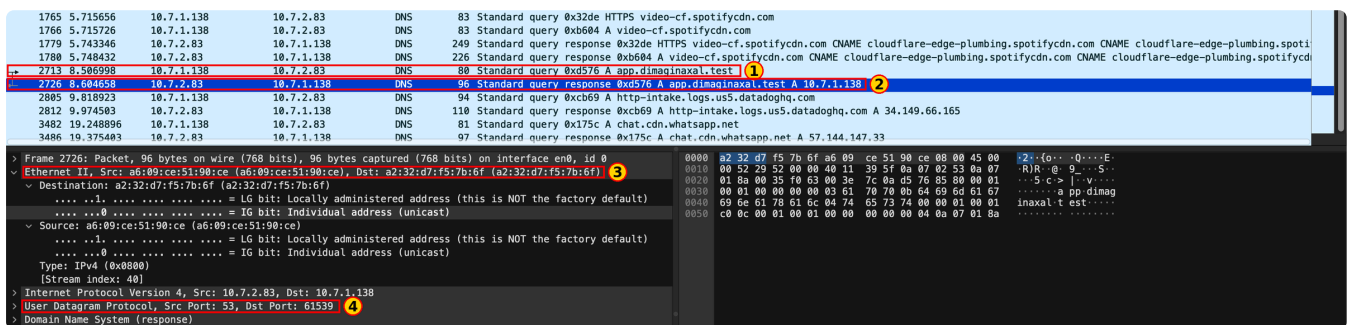


Figure 2. DNS response (frame 2726).

1. The original query.
2. The response carries A 10.7.1.138, which is Mac 2's own address.
3. The reply comes back from Mac 1's MAC to Mac 2's MAC.
4. Ports are now reversed: source 53, destination 61539.

The other rows in the capture are Spotify, WhatsApp and Datadog lookups from Mac 2. Mac 1 forwarded those upstream, which shows the `server=` lines in dnsmasq work too. Our own names never leave the LAN.

DNS only gives the client an address. No connection to the service exists yet at this point; that starts with TCP in the next step.

6. Task C: Two backend services

Each backend is one small Node.js file using the built-in `http` module, with no framework. Both have the same code except for the port and the letter they report.

	Backend A	Backend B
Machine	Mac 3 (Sibtain Ahmed Qureshi)	Mac 4 (Viraj Chafale)
Listens on	0.0.0.0:3001	0.0.0.0:3002
GET /	<code>{"message": "Dimaginaxal Backend A", "backend": "A", "status": "running"}</code>	same, with B
GET /api/status	<code>{"backend": "A", "status": "ok"}</code>	<code>{"backend": "B", "status": "ok"}</code>
Extra headers	X-Backend: A, Cache-Control: public, max-age=60	X-Backend: B, same Cache-Control

Binding to **0.0.0.0** matters. A server bound to 127.0.0.1 only accepts connections from the same machine, so nginx on Mac 2 could never reach it.

We tested each backend twice: locally, and from another Mac over the LAN.

```
$ curl -i http://10.7.12.144:3001/api/status
HTTP/1.1 200 OK
X-Backend: A
{"backend": "A", "status": "ok"}

$ curl -i http://10.7.16.85:3002/api/status
HTTP/1.1 200 OK
X-Backend: B
{"backend": "B", "status": "ok"}
```

Full source is in the Appendix.

7. Task D: nginx reverse proxy and load balancer on Mac 2

Mac 2 runs **nginx 1.31.6** and is the only machine clients talk to. The two backends sit in one upstream group:

```
upstream backend_pool {
    server 10.7.12.144:3001;    # Backend A
    server 10.7.16.85:3002;    # Backend B
}
```

Port 80 only redirects to HTTPS (301). Port 443 terminates TLS and passes the request to `backend_pool` over plain HTTP. No balancing method is named, so nginx uses its default, round-robin: each new request goes to the next server in the list.

The client never learns the backend addresses. It only knows `app.dimaginaxal.test`, which points to Mac 2. We can add, remove or move backends by editing nginx alone, with no change on any client.

Load balancing result, six requests in a row from a client:

```
$ for i in 1 2 3 4 5 6; do curl -s https://app.dimaginaxal.test/api/status;
echo; done
{"backend": "A", "status": "ok"}
{"backend": "B", "status": "ok"}
{"backend": "A", "status": "ok"}
{"backend": "B", "status": "ok"}
{"backend": "A", "status": "ok"}
{"backend": "B", "status": "ok"}
```

The answers alternate exactly A, B, A, B, A, B, which is what round-robin should give with two equal servers.

We also captured the hop behind nginx, from Mac 2 to Backend B:

No.	Time	Source	Destination	Protocol	Length	Info
114	1.251212	10.7.1.138	10.7.16.85	TCP	78	59385 → 3002 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=1201750994 TSecr=0 SACK_PERM
117	1.265748	10.7.16.85	10.7.1.138	TCP	78	3002 → 59385 [SYN, ACK, ECE] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=64 TSval=1159780607 TSecr=1201750994 SACK_PERM
118	1.265926	10.7.1.138	10.7.16.85	TCP	66	59385 → 3002 [ACK] Seq=1 Ack=1 Win=131776 Len=0 TSval=1201751009 TSecr=1159780607
119	1.265593	10.7.1.138	10.7.16.85	HTTP	239	GET /api/status HTTP/1.1
120	1.272097	10.7.16.85	10.7.1.138	TCP	66	3002 → 59385 [ACK] Seq=1 Ack=174 Win=131648 Len=0 TSval=1159780618 TSecr=1201751009
121	1.272100	10.7.16.85	10.7.1.138	TCP	66	[TCP Dup ACK 120#1] 3002 → 59385 [ACK] Seq=1 Ack=174 Win=131648 Len=0 TSval=1159780618 TSecr=1201751009
122	1.315755	10.7.16.85	10.7.1.138	HTTP/JSON	318	HTTP/1.1 200 OK, JSON (application/json)
123	1.315903	10.7.1.138	10.7.16.85	TCP	66	59385 → 3002 [ACK] Seq=174 Ack=253 Win=131584 Len=0 TSval=1201751059 TSecr=1159780659

Figure 3. nginx forwarding a request to Backend B, filter `tcp.port == 3001 || tcp.port == 3002`.

1. TCP three-way handshake: Mac 2 (port 59385) sends SYN to 10.7.16.85:3002, Backend B replies SYN-ACK, Mac 2 sends ACK.
2. nginx sends `GET /api/status HTTP/1.1` in plain text. This hop is inside our LAN and is not encrypted.
3. Backend B answers `HTTP/1.1 200 OK` with a JSON body.

4. The display filter used.

The red Dup ACK row (121) is a duplicate acknowledgement from Backend B. It did not affect the request, which completed in about 65 ms from SYN to response.

The same capture for Backend A (Mac 2 → 10.7.12.144:3001) was taken during testing, but the screenshot is not in this report yet.

8. Task E: HTTPS and TLS

TLS ends at nginx on Mac 2. Between the client and Mac 2 everything is encrypted; between Mac 2 and the backends it is plain HTTP (Figure 3).

We used **mkcert 1.4.4** to make a small local certificate authority (CA) on Mac 2 and sign a certificate for `app.dimaginaxal.test`, valid until 4 January 2029.

```
mkcert -install                # creates the local CA on Mac 2
mkcert app.dimaginaxal.test    # app.dimaginaxal.test.pem +
app.dimaginaxal.test-key.pem
```

The certificate and key were copied to `/opt/homebrew/etc/nginx/certs/`, and the key's permissions were tightened.

Making other Macs trust it. Mac 3 first refused the certificate, because it had never heard of our CA. We copied only the public `rootCA.pem` from Mac 2 and added it to Mac 3's system keychain:

```
sudo security add-trusted-cert -d -r trustRoot \
-k /Library/Keychains/System.keychain ~/rootCA.pem
```

After that, `curl https://app.dimaginaxal.test/api/status` on Mac 3 returned `{"backend":"A","status":"ok"}` with no `-k`, so certificate checking was fully on. The same command also passed on Mac 2.

The CA private key (`rootCA-key.pem`) and the server key never leave Mac 2. Anyone holding the CA key could sign a fake certificate that our Macs would trust.

Handshake steps, as we expect to see them for TLS 1.3:

1. **ClientHello**: the client lists its TLS versions and ciphers, and names the site in SNI (`app.dimaginaxal.test`).
2. **ServerHello**: nginx picks the version and cipher and sends its key share.
3. **Certificate**: nginx sends the mkcert certificate. In TLS 1.3 this is already encrypted, so Wireshark shows it only as encrypted handshake data.

4. **Finished:** both sides confirm, and Application Data (the HTTP request and response) follows.

We did capture a TLS 1.3 handshake from Mac 3 (10.7.12.144) to Mac 2 (10.7.1.138) with SNI `app.dimaginaxal.test`. That screenshot is not in this report yet and will be added to the evidence folder.

9. Task F: HTTP caching

Both backends send `Cache-Control: public, max-age=60`, and nginx passes it through to the client unchanged:

```
$ curl -I https://app.dimaginaxal.test/api/status
HTTP/1.1 200 OK
Server: nginx/1.31.6
Content-Type: application/json
X-Backend: B
Cache-Control: public, max-age=60
```

This tells any browser or shared cache that the response may be reused for 60 seconds without asking the server again.

The three cases the guideline asks us to tell apart:

Case	What happens	What you see
Fresh cache hit	Copy is under 60 s old, reused without any request	Browser shows "(disk cache)" or "(memory cache)"
Conditional request	Copy is stale; client asks "changed since?" using an ETag or date	<code>304 Not Modified</code> , no body
Full new request	No usable copy	<code>200 OK</code> with the full body

So far we have only shown the header. A cache hit and a 304 are not demonstrated yet. Our backends send no `ETag` or `Last-Modified`, so a 304 is not possible with the current code; adding an ETag is a small change we plan before the review.

10. Task G: Protocol flow in Wireshark

All captures were taken on Mac 2's Wi-Fi interface (en0). Mac 2 is the edge, so it sees both sides of every project request.

Ports in one request:

Step	Protocol	Client side	Server side
Name lookup	DNS over UDP	Mac 2, port 61539 (random)	Mac 1, port 53
Client to edge	TLS over TCP	client, random high port	Mac 2, port 443
Edge to backend	HTTP over TCP	Mac 2, port 59385 (random)	Mac 4, port 3002

The client side always picks a random high (ephemeral) port; the server side uses the fixed, known port. Source IP, source port, destination IP and destination port together name one connection (the socket pair).

TCP handshake on port 443. Our capture with filter `tcp.port == 443` was taken while Mac 2 was also browsing the internet, so most rows are outside traffic. We use one of those connections below to explain the handshake, because it shows every field clearly. It is **not** our own client-to-nginx connection.

No.	Time	Source	Destination	Protocol	Length	Info
2813	9.985153	10.7.1.138	34.149.66.165	TCP	78	61261 → 443 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=3018612317 TSecr=0 SACK_PERM
2814	9.989669	34.149.66.165	10.7.1.138	TCP	74	443 → 61261 [SYN, ACK, ECE] Seq=0 Ack=1 Win=28968 Len=0 MSS=1460 SACK_PERM TSval=2291010549 TSecr=3018612317 WS=64
2815	9.989878	10.7.1.138	34.149.66.165	TCP	66	61261 → 443 [ACK] Seq=1 Ack=1 Win=131776 Len=0 TSval=3018612322 TSecr=2291010549
2816	9.996761	10.7.1.138	34.149.66.165	TCP	1514	61261 → 443 [ACK] Seq=1 Ack=1 Win=131776 Len=1448 TSval=3018612329 TSecr=2291010549 [TCP PDU reassembled in 2817]
2817	9.996822	10.7.1.138	34.149.66.165	TLSv1.3	352	Client Hello (SN[=http-intake.logs.us5.datadoghq.com])
2818	10.001107	34.149.66.165	10.7.1.138	TCP	66	443 → 61261 [ACK] Seq=1 Ack=1449 Win=32000 Len=0 TSval=2291010560 TSecr=3018612329
2819	10.001110	34.149.66.165	10.7.1.138	TCP	66	443 → 61261 [ACK] Seq=1 Ack=1735 Win=34816 Len=0 TSval=2291010560 TSecr=3018612329
2820	10.001245	10.7.1.138	34.149.66.165	TCP	66	443 → 61261 [ACK] Seq=1 Ack=1735 Win=34816 Len=0 TSval=2291010560 TSecr=3018612329
2821	10.422720	34.149.66.165	10.7.1.138	TLSv1.3	1466	Server Hello, Change Cipher Spec
2831	10.422724	34.149.66.165	10.7.1.138	TCP	1514	443 → 61261 [ACK] Seq=1401 Ack=1735 Win=34816 Len=1448 TSval=2291010850 TSecr=3018612329 [TCP PDU reassembled in 2832]
2832	10.422726	34.149.66.165	10.7.1.138	TLSv1.3	711	Application Data
2833	10.422728	34.149.66.165	10.7.1.138	TCP	711	[TCP Retransmission] 443 → 61261 [PSH, ACK] Seq=2849 Ack=1735 Win=34816 Len=645 TSval=2291010899 TSecr=3018612329 [TCP PDU reassembled in 2834]
2834	10.422730	10.7.1.138	34.149.66.165	TLSv1.2	569	Application Data
2835	10.422732	10.7.1.138	34.149.66.165	TCP	66	443 → 61261 [ACK] Seq=1 Ack=2 Win=149 Len=0 TSval=127505402 TSecr=500635816
2836	10.422950	10.7.1.138	34.149.66.165	TCP	66	61261 → 443 [ACK] Seq=1735 Ack=3494 Win=128320 Len=0 TSval=3018612753 TSecr=2291010850
2837	10.423071	10.7.1.138	34.149.66.165	TCP	78	443 → 61261 [ACK] Seq=1 Ack=1461 Win=2041 Len=0 TSval=186811877 TSecr=602775208
2838	10.423142	10.7.1.138	10.7.1.138	TCP	66	443 → 443 [ACK] Seq=1 Ack=1461 Win=2041 Len=0 TSval=186811877 TSecr=602775208
2839	10.439200	10.7.1.138	34.149.66.165	TLSv1.3	1514	Change Cipher Spec, Application Data

Frame 2813: Packet, 78 bytes on wire (624 bits), 78 bytes captured (624 bits) on interface en0, id 0
 Ethernet II, Src: a2:32:d7:f5:7b:6f (a2:32:d7:f5:7b:6f), Dst: RuckusWireless-6f:38:ee (c0:c5:20:6f:38:ee)
 Destination: RuckusWireless-6f:38:ee (c0:c5:20:6f:38:ee)
 Type: IPv4 (0x0800)
 [Stream index: 0]
 Internet Protocol Version 4, Src: 10.7.1.138, Dst: 34.149.66.165
 Transmission Control Protocol, Src Port: 61261, Dst Port: 443, Seq: 0, Len: 0

Figure 4. TCP three-way handshake on port 443 (Mac 2 to an outside server, 34.149.66.165).

1. SYN (Seq=0), SYN-ACK (Seq=0, Ack=1), ACK (Seq=1, Ack=1). Each side's Ack is the other side's Seq + 1, which is how both confirm the other's starting number. The whole handshake took about 5 ms.
2. Only after the handshake does TLS start: the ClientHello here names `http-intake.logs.us5.datadoghq.com`, which is how we know this is not project traffic.
3. Ethernet destination is the Wi-Fi router (RuckusWireless, `c0:c5:20:6f:38:ee`). In Figure 1 the destination was Mac 1's own MAC. A frame goes straight to a host on the same subnet, but to the gateway for anything outside it.
4. Client port 61261, server port 443.

Other things the capture shows about TCP reliability: retransmissions and duplicate ACKs (red and black rows). A duplicate ACK means the receiver is still waiting for a missing segment; the sender resends it. Wi-Fi loss makes these common.

Still to capture: a clean SYN / SYN-ACK / ACK between a client Mac and 10.7.1.138:443, best with filter `ip.addr == 10.7.1.138 && tcp.port == 443` and no other apps running.

11. Phase 1 status

The Phase 1 gate is met: a client resolves `app.dimaginaxal.test` through Mac 1, connects over trusted HTTPS, and gets answers from both backends. The gaps below are evidence, not missing setup, except the 304 demo and the failure tests.

Task	Status	Shown in this report	Still to add before review
A. Private LAN	Done	IP/MAC table, ping results	Ping with Mac 3 and Mac 4 as source
B. Private DNS	Done	dnsmasq config, dig result, Figures 1 and 2	Terminal screenshot of <code>dig</code>
C. Two backends	Done	Source, curl output from LAN	
D. Reverse proxy + load balancing	Done	Upstream config, A/B/A/B/A/B result, Figure 3	Wireshark screenshot for Backend A (port 3001)
E. HTTPS / TLS	Partly done	mkcert setup, CA trusted on Mac 3, curl without -k	TLS capture screenshot with SNI <code>app.dimaginaxal.test</code> ; CA trust on Mac 1 and Mac 4; browser padlock screenshot
F. HTTP caching	Partly done	Cache-Control header via curl -I	Add ETag, show a 304 or a browser cache hit
G. Protocol flow	Partly done	DNS, edge-to-backend TCP + HTTP, port table	Clean client → 10.7.1.138:443 handshake; saved .pcapng file
Failure demos (Section 6.3)	Not started		All five: wrong DNS server, wrong DNS record, one backend down, both backends down (expect 502), wrong port

For "one backend down": nginx by default retries the next server when a connection is refused, so requests should keep working through the other backend. We still need to test and record it.

12. What we learned

- **DNS and connections are separate.** In our capture the lookup finished over UDP in about 0.1 s, before any TCP packet to the service. DNS only says where to go; it does not open the connection.
- **Same subnet means direct delivery.** Our DNS packets went Mac 2 → Mac 1 by MAC address, while internet traffic went to the router's MAC. We had read this in class; seeing two different destination MACs in one capture made it concrete.
- **TLS is only as strong as trust.** The certificate itself worked from the start. What failed on Mac 3 was trust in our CA, and installing one public file fixed it. Keeping the CA key private is what keeps this safe.
- **The edge hides the backends.** Clients only know one name and one IP. nginx decides who answers, which is why we could prove load balancing with a simple loop of curl calls.
- **Wi-Fi is noisy.** The 25% ping loss and the retransmissions in our captures showed TCP doing its job: lost data was resent and the requests still completed.

Next (Phase 2): backup DNS on a second Mac, short TTLs and a record change, firewall rules so only Mac 2 can reach ports 3001/3002, failover tests, and moving the edge by DNS.

Appendix

A. dnsmasq.conf (Mac 1, ~/network-project/dnsmasq.conf)

```
port=53
listen-address=127.0.0.1,10.7.2.83
bind-interfaces

address=/app.dimaginaxal.test/10.7.1.138
address=/api.dimaginaxal.test/10.7.1.138

no-resolv
server=1.1.1.1
server=8.8.8.8

cache-size=1000
```

Started with `sudo dnsmasq -C ~/network-project/dnsmasq.conf -d` (foreground, so the log is visible).

B. nginx.conf (Mac 2, /opt/homebrew/etc/nginx/nginx.conf)

```

worker_processes 1;
events { worker_connections 1024; }

http {
    include mime.types;
    default_type application/octet-stream;
    sendfile on;
    keepalive_timeout 65;

    upstream backend_pool {
        server 10.7.12.144:3001;
        server 10.7.16.85:3002;
    }

    server {
        listen 80;
        server_name app.dimaginaxal.test;
        return 301 https://$host$request_uri;
    }

    server {
        listen 443 ssl;
        server_name app.dimaginaxal.test;

        ssl_certificate
/opt/homebrew/etc/nginx/certs/app.dimaginaxal.test.pem;
        ssl_certificate_key
/opt/homebrew/etc/nginx/certs/app.dimaginaxal.test-key.pem;

        location / {
            proxy_pass http://backend_pool;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
        }
    }
}

```

Checked with `nginx -t` (syntax ok). A copy of the original is kept as `nginx.conf.backup`.

C. Backend A (Mac 3, ~/dimaginaxal/backend-a/server.js)

Backend B is the same file with `PORT = 3002` and `"B"` in place of `"A"`.

```

const http = require("http");
const PORT = 3001;

const server = http.createServer((req, res) => {
  res.setHeader("Content-Type", "application/json");
  res.setHeader("X-Backend", "A");
  res.setHeader("Cache-Control", "public, max-age=60");

  if (req.url === "/api/status") {
    res.writeHead(200);
    res.end(JSON.stringify({ backend: "A", status: "ok" }));
    return;
  }
  if (req.url === "/") {
    res.writeHead(200);
    res.end(JSON.stringify({ message: "Dimaginaxal Backend A", backend:
"A", status: "running" }));
    return;
  }
  res.writeHead(404);
  res.end(JSON.stringify({ error: "Not Found" }));
});

server.listen(PORT, "0.0.0.0", () => {
  console.log(`Backend A running on port ${PORT}`);
});

```

Run with `node server.js` inside the backend folder.

D. Wireshark filters used

Filter	Shows
<code>dns</code>	All DNS queries and answers
<code>tcp.port == 443</code>	HTTPS connections
<code>tcp.flags.syn == 1 && tcp.port == 443</code>	Only connection openings on 443
<code>tls</code>	TLS handshakes and encrypted records
<code>tcp.port == 3001 tcp.port == 3002</code>	nginx to backend traffic